P0846R0
John Spicer <jhs@edg.com>
Audience: CWG
2017-11-08

P0846R0: ADL and Function Templates that are not Visible

Introduction

In Toronto (7/2017) core reviewed Robert Haberlach's D0389R1 "template keyword in unqualified-ids". Core agreed with that the problem presented but disagreed with the intended solution.

Instead of requiring the user to use the `template` keyword, a revision to the lookup rules was proposed so that a name for which a normal lookup produces either no result or finds one or more functions and that is followed by a a "<" would treated as if a function template name had been found and would cause ADL to be performed.

This proposal was brought to the Evolution group in Toronto and it received strong support (12 | 14 | 0 | 0 | 0).

It was observed that this change could change code where you have an overloaded < operator that accepts a function as the left-hand operand. This case was considered as pathological case not likely enough to be of concern.

Wording changes

Change 6.4.1 [basic.lookup.unquql] paragraph 3:

> The lookup for an unqualified name used as the *postfix-expression* of a function call is described in 6.4.2. [Note: For purposes of determining (during parsing) whether an expression is a *postfix-expression* for a function call, the usual name lookup rules apply. **In some cases a name followed by < is treated as a *template-name* even though name lookup did not find a template-name (see 17.2). For example,**
>
> ```
> int h;
> void g();
> namespace N {
>         struct A {};
>         template <class T> int f(T);
>         template <class T> int g(T);
>         template <class T> int h(T);
> }
>
> int x = f<N::A>(N::A());  // OK: lookup of f finds nothing,
>                           // f treated as template name
> int y = g<N::A>(N::A());  // OK: lookup of g finds a function,
>                           // g treated as template name
> int z = h<N::A>(N::A());  // error: "h<" does not begin a template-id
> ```
>
> The rules in 6.4.2 have no effect on the syntactic interpretation of an expression. For example,

Change 17.2 [temp.names] paragraph 2 and 3:

For a *template-name* to be explicitly qualified by the template arguments, the name must be ~~known~~ **considered** to refer to a template. **[Note: Whether a name actually refers to a template cannot be known in some cases until after argument dependent lookup is done [6.4.2]. -- end note] A name is considered to refer to a template if name lookup finds a *template-name* or a overload set that contains a function template. A name is also considered to refer to a template if it is an *unqualified-id* followed by a < and name lookup finds either one or more functions or finds nothing.**

~~After name lookup (6.4) finds that a name is~~ **When a name is considered to be** a *template-name* ~~or that an operator-function-id or a literal- operator-id refers to a set of overloaded functions any member of which is a function template~~, ~~if this~~ **and it** is followed by a <, the < is always taken as the delimiter of a template-argument-list and never as the less-than operator.

Change 17.3 [temp.arg] paragraph 7:

When ~~the template~~ **name lookup for the name** in a template-id ~~is an overloaded function template~~ **finds an overload set**, both non-template functions in the overload set and function templates in the overload set ...

Add a new section to Annex C.6:

C.6.X [temp.name]

Change: A *unqualified-id* that is followed by a < and for which name lookup finds nothing or finds a function will be treated as a *template-name* in order to potentially cause argument dependent lookup to be performed.

Rationale: It was problematic to call a function template with an explicit template argument list via argument dependent lookup because of the need to have a template with the same name visible via normal lookup.

Effect on original feature: Previously valid code that uses a function name as the left operand of a < operator would become ill-formed.

```
struct A {};
bool operator <(void (*fp)(), A);
void f(){}
int main() {
  A a;
  f < a;  // ill-formed; previously well-formed
  (f) < a;  // still well formed
}
```